PROJECT-BASED LEARNING

PHASE-I EVALUATION

Smart Traffic Flow Optimization & Emergency Preemption Engine

An Interactive 2D Simulation Game in C++

Team: FlowOptimizers — **Team ID:** DSCPP-321-III

Department of Computer Science & Engineering
Graphic Era (Deemed to be University), Dehradun
Academic Session 2026–27

Team & Mentor Details

Team Details

Team Name:	FlowOptimizers
Team ID:	DSCPP-321-III
Semester:	3rd Semester
Section:	C (ML2, ML3)
Domain:	Traffic Automation & 2D Game

Team Members

1. **Yashvardhan Singh (Lead)** (2510015374)
2. **Arsh Singh** (2510370430)
3. **Sambhav Jaiswal** (2510370170)

Faculty Mentor

Mentor Name: Mr. Kamal Kumar Gola **Interactions Completed:** 2

Problem Statement & Motivation

The Core Problem

Anyone driving in a city knows the frustration of sitting at a red light while the cross-street is empty. Static timers run blindly, burning fuel and stalling commuters. Worse, they trap ambulances in long queues. We built a playable 2D game in C++ to prove how fundamental data structures can make lights react to real queues and clear immediate paths for emergency vehicles.

Everyday Issues We Address

- Timers switch based on clocks, not actual traffic.
- Ambulances get stuck behind lines of waiting cars.
- No quick way to divert cars when an intersection backs up.

How Our Game Solves It

- Greener lights stay on longer for busier lanes.
- Priority heaps instantly clear paths for ambulances.
- An interactive visual test you can actually drive and see.

Objectives

What We Set Out to Build

1. **Playable 2D Grid:** Build a clean top-down road network using C++ and SFML/Raylib.
2. **Smart Green Lights:** Allocate green light times based on real queue density (using Passenger Car Equivalents) so heavier lanes clear first.
3. **Ambulance Priority:** Use a Min-Heap priority queue to instantly flip signals green and flush queues ahead of emergency vehicles.
4. **Side-by-Side Mode Toggle:** Give the user an in-game switch (Old Timers vs. Smart Mode) to watch wait times drop in real time.

Core Goal: A hands-on 2D game that makes the real-world value of data structures obvious and visual.

Proposed Solution

How the Simulation Works

- **Player Vehicle:** Drive a car or emergency ambulance across the grid using keyboard arrow keys.
- **Autonomous Traffic:** AI cars spawn, navigate waypoints, and brake smoothly using simple raycast sensors.
- **Lane Management:** Each road acts as a FIFO queue tracking vehicle sizes and delays.

Interactive Demo Modes

- **Mode 1 (Fixed Timer):** Standard fixed countdowns; cars quickly pile up into visible gridlock.
- **Mode 2 (Smart DSA Engine):** Signals favor crowded lanes and instantly clear ambulance corridors.
- **Live Dashboard:** An on-screen HUD displays live average delays and vehicle throughput.

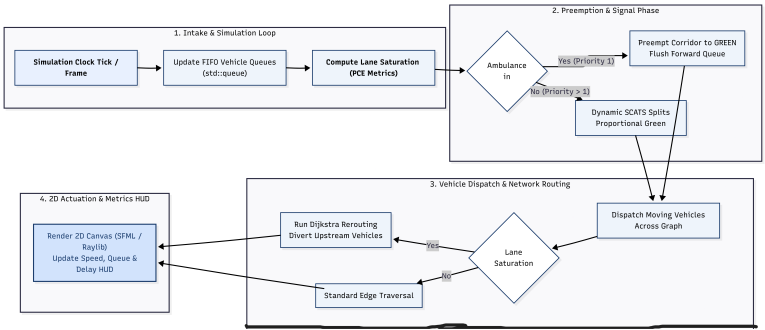
Using a game format makes algorithmic efficiency intuitive rather than just rows of console numbers.

Integration of PBL Course Concepts

Course Area	Concepts Applied	Where It Lives in the Game
Data Structures & Algorithms (CS201)	• Adjacency List Graph • FIFO Queues (`std::queue`) • Min-Heap (`std::priority_queue`) • Dijkstra's Shortest Path	• Road network nodes and lanes • Physical lines of cars at stop lines • Instant ambulance priority detection • Rerouting cars around blocked junctions
Object-Oriented Programming (C++)	• Class Inheritance & Polymorphism • Encapsulation & Clean Interfaces • Game Loop & Delta Timing	• Base `Vehicle` → `Car`, `Bus`, `Ambulance` • Modular `Lane`, `Intersection`, and `Signal` • Smooth frame-by-frame 2D rendering

Course Integration: Core DSA structures handle traffic logic, wrapped neatly inside an OOP game engine.

System Architecture / Workflow



1. Signal Evaluation

- The priority heap checks for approaching ambulances ($O(1)$ lookup).
- Dynamic splits scale green lights to match queue density.

2. Vehicle Movement & HUD

- Cars move forward during green lights via graph edges.
- Clogged lanes trigger alternate routes; live HUD updates.

Technology Stack & Methodology

Technology Stack

- **Core Language:** C++ (C++17 standard)
- **Libraries:** C++ Standard Template Library (STL)
- **Graphics/Windowing:** SFML 2.6 / Raylib
- **Build Tool:** CMake
- **Development:** VS Code / Cursor, Git & GitHub

How We're Building It

1. **Core Logic First:** Implement queue and heap logic in terminal prototypes.
2. **2D Canvas:** Add road tiles, car sprites, and player controls.
3. **Signal Engine:** Connect the adaptive timing and preemption logic.
4. **Benchmarking:** Add the live HUD to measure real delay drops.

Using native C++ and SFML keeps the project lightweight, dependency-free, and centered on DSA.

Initial Progress & Team Contribution

What We've Done So Far

- Finalized project scope and simplified system workflow.
- Designed OOP vehicle and lane class hierarchies.
- Verified ambulance preemption logic in test scripts.
- Created the basic 2D SFML window and rendering loop.

Individual Contributions

Member	Role & Main Responsibilities
Yashvardhan Singh	Overall architecture, signal timing math, and Dijkstra rerouting.
Arsh Singh	OOP vehicle classes, lane FIFO queues, and Min-Heap priority preemption.
Sambhav Jaiswal	2D SFML rendering window, player vehicle physics, and HUD metrics.

Roadmap, Expected Outcomes & References

Next Steps

1. **Phase-I (Current):** Problem formulation, OOP design, and logic prototypes.
2. **Phase-II:** 2D road grid, player controls, and AI vehicle pathing.
3. **Phase-III:** Live benchmarking HUD and comparative test runs.

Expected Outcomes

- An interactive, playable 2D traffic game.
- Noticeable drop in vehicle wait times in smart mode.
- Zero delay for ambulances passing through.

Key References

1. Lowrie, P. R. (1990). *SCATS: Sydney Coordinated Adaptive Traffic System*.
2. Cormen, T. H., et al. (2009). *Introduction to Algorithms* (Graph Algorithms & Heaps).
3. SFML Team. (2024). *SFML Documentation*. `https://www.sfml-dev.org`

Thank You

Questions & Discussion